SISTEMA DE SERVICIO COMUNITARIO · IUNAV

Manual de instalación local sin kit

Guía paso a paso para instalar Node.js y PostgreSQL, configurar el proyecto y levantar la API y la web en Windows 10/11.

Dirigido a: tutor evaluador y personal que instalará el sistema.

Versión del manual: 1.0 · octubre de 2026

Tiempo estimado: 30 a 45 minutos la primera vez; 3 minutos después.

Resultado esperado: sistema funcionando en `http://localhost:3000`.

> **Este manual no usa el kit automático.** No ejecute `local\instalar.cmd`, `iniciar.cmd` ni `detener.cmd`. Aquí se instala PostgreSQL aparte y se levanta el proyecto con comandos de Node.js.

Cómo usar este manual

Siga los pasos en orden. Cada paso indica qué hacer, qué comando escribir y qué debería ver. No avance al siguiente paso hasta que el anterior haya funcionado. Si algo falla, vaya a la sección **13. Si algo falla** antes de repetir la instalación desde cero.

Antes de empezar

- Tenga a mano la contraseña del usuario `postgres` que va a crear. La necesitará varias veces.
- Use una cuenta de Windows con permisos de administrador para instalar Node.js y PostgreSQL.
- Necesita conexión a internet solo durante la instalación. Después el sistema funciona sin internet.
- Libere unos 3 GB de disco y cierre programas que usen los puertos 3000, 4000 o 5432.
- No instale el proyecto dentro de una carpeta con acentos o espacios raros. Se recomienda **`C:\IUNAV\servicio-comunitario-iunav`**.

Lista de verificación

Marque cada casilla a medida que avanza:

- [ ] Node.js instalado y `node -v` responde.
- [ ] PostgreSQL 17 instalado y el servicio está corriendo.
- [ ] La base de datos `app_iunav` creada.
- [ ] **`server\.env`** configurado con la contraseña correcta y `DB_SSL=false`.
- [ ] Esquema aplicado con `npm run db:schema`.
- [ ] Datos de ejemplo cargados con `npm run db:seed -- --confirm-seed`.
- [ ] API respondiendo en `http://localhost:4000/api/health`.
- [ ] Web abierta en `http://localhost:3000`.
- [ ] Entró con el usuario administrador.
- [ ] Probó los cuatro roles.

> **El repositorio es público, pero el archivo server\.env es privado.**
> Ese archivo guarda la contraseña de su PostgreSQL y el secreto de las sesiones. No lo comparta, no lo suba a GitHub y no lo envíe por correo.

> **Si ya tiene PostgreSQL instalado**
> No hace falta reinstalarlo. Anote la contraseña del usuario `postgres` y el puerto (normalmente 5432), y empiece en el paso 4 para crear la base. Si el puerto es distinto, ajuste `DATABASE_URL` en el paso 5. Si también tiene Node.js, puede saltar el paso 1.

Contenido

1. Qué va a instalar y cómo funciona
2. Requisitos de la computadora
3. Paso 1: instalar Node.js
4. Paso 2: instalar PostgreSQL
5. Paso 3: descargar el proyecto
6. Paso 4: crear la base de datos
7. Paso 5: configurar la API
8. Paso 6: instalar dependencias, esquema y datos
9. Paso 7: levantar la API
10. Paso 8: levantar la web
11. Paso 9: verificar con los cuatro roles
12. Uso diario: iniciar, detener y reiniciar
13. Si algo falla
14. Reiniciar los datos y desinstalar
15. Anexo: estructura, puertos y comandos

1. Qué va a instalar y cómo funciona

El sistema tiene tres piezas que se ejecutan en su propia computadora:

Pieza	Qué es	Dirección
Web	La interfaz que se abre en el navegador. Está hecha con Next.js y React.	`http://localhost:3000`
API	El servidor que valida usuarios, permisos y datos. Está hecho con Express y TypeScript.	`http://localhost:4000`
Base de datos	PostgreSQL, donde viven usuarios, grupos, bitácoras y servicios.	`localhost:5432`

El recorrido de una petición es este:

```
Navegador
  http://localhost:3000
          |
          v
  Web (Next.js)  :3000
          |
          |  /api/*
          v
  API (Express)  :4000
          |
          v
  PostgreSQL     :5432
```

Qué significa "sin kit"

El proyecto incluye una carpeta **`local\`** con un kit que instala un PostgreSQL embebido y arranca todo con doble clic. **Este manual no usa ese kit.** Aquí usted instala Node.js y PostgreSQL con sus instaladores oficiales, crea la base de datos y ejecuta los comandos del proyecto. Es la instalación manual, paso a paso.

> **No ejecute los archivos .cmd de la carpeta local**
>
> Si ve `instalar.cmd`, `iniciar.cmd` o `detener.cmd`, déjelos quietos. Este manual sigue otro camino: PostgreSQL instalado aparte y dos terminales con `npm run dev`.

Qué va a quedar en su computadora

- Node.js 24 LTS y npm.
- PostgreSQL 17 y pgAdmin 4.
- El proyecto en **`C:\IUNAV\servicio-comunitario-iunav`**.
- La base `app_iunav` con las tablas y los datos de ejemplo.
- La web y la API corriendo mientras las terminales estén abiertas.

2. Requisitos de la computadora

Requisito	Mínimo	Recomendado
Sistema operativo	Windows 10 de 64 bits	Windows 11 de 64 bits
Memoria RAM	4 GB	8 GB o más
Espacio libre	3 GB	5 GB
Permisos	Administrador para instalar	Administrador
Internet	Solo para instalar	Solo para instalar
Puertos libres	3000, 4000 y 5432	Los mismos

Programas que se van a instalar

- **Node.js 24 LTS** — mínimo 20.9. Incluye npm, que se usa para instalar dependencias y arrancar el proyecto.
- **PostgreSQL 17** — base de datos. El instalador incluye pgAdmin 4 y las herramientas de línea de comandos.
- **Git** OPCIONAL — solo si va a descargar el proyecto con `git clone`. También puede usar el ZIP de GitHub.

Cómo saber si ya tiene algo instalado

Abra PowerShell y ejecute estos comandos. Si responden con una versión, ya está instalado:

```
node -v
npm -v
git --version
```

Para PostgreSQL, abra el menú Inicio y busque **SQL Shell (psql)** o **pgAdmin 4**. También puede revisar **Servicios** y buscar un servicio cuyo nombre empiece por `postgresql`.

> **No instale dos veces PostgreSQL**
>
> Si ya lo tiene, use el que existe. Solo necesita la contraseña del usuario `postgres` y el puerto. Si no la recuerda, es mejor reinstalarlo o pedir ayuda antes de seguir.

Revise que los puertos estén libres

En PowerShell:

```
netstat -ano | findstr :3000
netstat -ano | findstr :4000
netstat -ano | findstr :5432
```

Si algún comando devuelve una línea, hay un programa usando ese puerto. Cierre ese programa antes de continuar. En la sección 13 se explica cómo identificar y cerrar el proceso si es necesario.

3. Paso 1: instalar Node.js

Node.js es el programa que ejecuta la API y las herramientas de la web. Instale la versión **LTS** (soporte a largo plazo), que hoy es la 24.

Opción A: instalador gráfico (recomendada)

1. Abra `https://nodejs.org/es/download` en el navegador.
2. Descargue el instalador **LTS para Windows (x64)**.
3. Ejecute el archivo descargado y acepte los valores por defecto. Deje marcada la opción que agrega Node al PATH.
4. Al terminar, cierre todas las ventanas de PowerShell o CMD que tuviera abiertas y abra una nueva. El PATH recién se actualiza en las terminales nuevas.

Opción B: con winget

Si prefiere la terminal, abra PowerShell como administrador y ejecute:

```
winget install OpenJS.NodeJS.LTS
```

Al terminar, cierre y vuelva a abrir PowerShell.

Verificar

En una terminal nueva:

```
node -v
npm -v
```

Debe ver algo parecido a esto:

```
v24.19.0
11.6.0
```

> **La versión exacta puede variar**
>
> Lo importante es que Node sea **24.x** o, como mínimo, **20.9**. Si ve una versión 18 o anterior, instale la LTS actual antes de continuar.

> **Si PowerShell dice "node no se reconoce como un comando"**
>
> Cierre la terminal, abra una nueva y pruebe otra vez. Si sigue igual, reinstale Node.js asegurándose de que la opción *Add to PATH* esté marcada.

4. Paso 2: instalar PostgreSQL

PostgreSQL guarda todos los datos del sistema. Se instala una sola vez y queda funcionando como un servicio de Windows.

Descargar e instalar

1. Abra `https://www.postgresql.org/download/windows/` y entre al instalador de **EDB**.
2. Descargue la versión **17.x** para Windows de 64 bits.
3. Ejecute el instalador con permisos de administrador.
4. Deje la carpeta de instalación por defecto: **`C:\Program Files\PostgreSQL\17`**.
5. Seleccione como mínimo estos componentes:
   - **PostgreSQL Server**
   - **pgAdmin 4**
   - **Command Line Tools**
6. Deje la carpeta de datos por defecto y escriba una **contraseña para el usuario postgres**.
7. Deje el puerto **5432** (a menos que ya esté ocupado; en ese caso use 5433 y anótelo).
8. Deje la configuración regional por defecto y continúe.
9. Al final, puede desmarcar **Stack Builder**; no hace falta para este proyecto.

> **Anote la contraseña del usuario postgres**
>
> Se usa en el paso 5 dentro de **`server\.env`**. Para evitar problemas, use una contraseña de letras y números, sin `@`, `:`, `/`, `#` ni `%`. Si usa esos caracteres, tendrá que codificarlos en la URL de conexión.

Verificar que quedó instalado

1. Abra el menú Inicio y busque **SQL Shell (psql)**.
2. Presione `Enter` para aceptar Servidor, Base de datos, Puerto y Usuario por defecto.
3. Escriba la contraseña de `postgres`. Al escribir no se ven los caracteres; es normal.
4. Debe aparecer el cursor `postgres=#`. Escriba:

```
SELECT version();
\q
```

La respuesta debe empezar por `PostgreSQL 17`. Si el comando `psql` no se reconoce desde PowerShell, no pasa nada: use siempre **SQL Shell** del menú Inicio, o la ruta completa:

```
& "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres
```

El servicio de PostgreSQL

El instalador crea un servicio llamado `postgresql-x64-17` que arranca automáticamente con Windows. Puede comprobarlo en **Servicios** o con:

```
Get-Service postgresql*
```

Debe decir `Running`. Si está detenido, haga clic derecho y elija **Iniciar**.

5. Paso 3: descargar el proyecto

Descargue el proyecto en una carpeta fácil de encontrar. Para que los comandos de este manual funcionen tal cual, use esta ruta:

```
C:\IUNAV\servicio-comunitario-iunav
```

Opción A: con Git (recomendada)

Si instaló Git, abra PowerShell y ejecute:

```
New-Item -ItemType Directory -Force C:\IUNAV
cd C:\IUNAV
git clone https://github.com/JesusQuilleli/servicio-comunitario-iunav.git
cd servicio-comunitario-iunav
```

Opción B: con el ZIP de GitHub

1. Abra `https://github.com/JesusQuilleli/servicio-comunitario-iunav`.
2. Haga clic en el botón verde **Code** y luego en **Download ZIP**.
3. Descomprima el archivo dentro de **`C:\IUNAV`**.
4. La carpeta se llamará algo como **`servicio-comunitario-iunav-master`**. Renómbrela a **`servicio-comunitario-iunav`**.
5. Abra PowerShell dentro de esa carpeta o ejecute `cd C:\IUNAV\servicio-comunitario-iunav`.

Verificar la descarga

Dentro de la carpeta del proyecto, ejecute:

```
Get-ChildItem
```

Debe ver, entre otras, estas carpetas y archivos:

```
docs\
local\        <- kit automático: NO se usa en este manual
server\       <- API
web\          <- interfaz web
README.md
```

> **No use la carpeta local**
>
> Es el kit automático. En este manual todo se hace desde **`server\`** y **`web\`** con comandos de npm.

> **Si no tiene Git o no puede clonarlo**
>
> Pida al responsable del proyecto que le entregue el PDF y un ZIP del proyecto, o que le dé acceso al repositorio. El ZIP debe incluir **`server\`**, **`web\`** y **`docs\`**, pero no debe incluir **`server\.env`** ni **`node_modules\`**.

6. Paso 4: crear la base de datos

El proyecto trae el esquema de las tablas, pero **no crea la base de datos**. Hay que crear una base vacía llamada `app_iunav`. Hágalo con pgAdmin o con psql.

Opción A: con pgAdmin 4 (recomendada)

1. Abra **pgAdmin 4** desde el menú Inicio.
2. En el panel izquierdo, haga clic en **Servers** y luego en **PostgreSQL 17**.
3. Escriba la contraseña del usuario `postgres`.
4. Haga clic derecho sobre **Databases** y elija **Create** > **Database...**.
5. En **Database**, escriba `app_iunav`.
6. Deje el propietario (*Owner*) en `postgres`.
7. Haga clic en **Save**.
8. Verifique que `app_iunav` aparece en la lista de bases de datos.

Opción B: con SQL Shell (psql)

Abra **SQL Shell (psql)**, acepte los valores por defecto y escriba:

```
CREATE DATABASE app_iunav;
\q
```

La respuesta debe ser `CREATE DATABASE`.

Base opcional para pruebas automáticas

Si más adelante quiere ejecutar la suite de pruebas del proyecto, cree también una base llamada `app_iunav_test`. No es necesaria para usar el sistema:

```
CREATE DATABASE app_iunav_test;
```

> **La base debe llamarse exactamente app_iunav**
>
> El archivo **`server\.env`** apunta a ese nombre. Si usa otro, la API no encontrará las tablas y responderá con errores.

> **Si pgAdmin no conecta**
>
> Revise que el servicio `postgresql-x64-17` esté en `Running` y que esté usando el puerto correcto. Si cambió la contraseña, use la nueva. Si no la recuerda, es más rápido reinstalar PostgreSQL que adivinarla.

7. Paso 5: configurar la API

La API necesita un archivo **`server\.env`** con la conexión a PostgreSQL y un secreto para las sesiones. El proyecto incluye una plantilla llamada **`.env.example`**.

Crear el archivo

En PowerShell:

```
cd C:\IUNAV\servicio-comunitario-iunav\server
Copy-Item .env.example .env
notepad .env
```

Se abrirá el Bloc de notas. Deje el archivo así, cambiando solo lo que se indica:

```
DATABASE_URL=postgresql://postgres:TU_CLAVE@localhost:5432/app_iunav
DB_SSL=false
JWT_SECRET=PEGA_AQUI_EL_SECRETO_QUE_GENERASTE
JWT_EXPIRES_IN=8h
NODE_ENV=development
PORT=4000
API_HOST=127.0.0.1
FRONTEND_URL=http://localhost:3000
UPLOAD_DIR=uploads
MAX_EVIDENCIA_MB=10
MAX_DOCUMENTO_MB=25
RATE_LIMIT_LOGIN=10
RATE_LIMIT_LOGIN_VENTANA_MS=900000
```

Qué significa cada valor importante

Variable	Qué poner
`DATABASE_URL`	Reemplace `TU_CLAVE` por la contraseña real de `postgres`. Si su PostgreSQL usa el puerto 5433, cámbielo aquí.
`DB_SSL`	Debe quedar exactamente en `false`. En local no hay certificado TLS; si se deja en `true`, la conexión falla.
`JWT_SECRET`	Una cadena larga y aleatoria. Use el comando de abajo para generarla.
`PORT`	4000. Es el puerto de la API.
`API_HOST`	127.0.0.1. La API solo escucha en este equipo.
`FRONTEND_URL`	http://localhost:3000. Debe coincidir con la dirección donde abre la web.

Generar el JWT_SECRET

En otra terminal, o antes de editar el archivo, ejecute:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Copie la salida completa (64 caracteres) y péguela después de `JWT_SECRET=`. No use la misma clave en dos instalaciones distintas.

Si la contraseña tiene caracteres especiales

En una URL, algunos caracteres deben codificarse. Si su contraseña es `MiClave#2026`, la URL debe llevar `MiClave%232026`. Lo más simple es usar una contraseña de solo letras y números. Ejemplos:

Carácter	Se escribe como
`@`	`%40`
`:`	`%3A`
`/`	`%2F`
`#`	`%23`
`%`	`%25`

> **No use comillas ni espacios**
>
> Escriba `DB_SSL=false`, no `DB_SSL = "false"`. Guarde el archivo con `Ctrl` + `S` y cierre el Bloc de notas. No suba este archivo a GitHub.

> **Si se equivoca, puede volver a empezar**
>
> Borre **`server\.env`**, copie otra vez **`.env.example`** y repita este paso. No hace falta desinstalar nada.

8. Paso 6: instalar dependencias, esquema y datos

Ahora se descargan las librerías del proyecto, se crean las tablas y se cargan los datos de ejemplo. Todo se ejecuta desde la carpeta **`server\`**.

1. Instalar dependencias

```
cd C:\IUNAV\servicio-comunitario-iunav\server
npm ci
```

Este paso descarga las librerías y puede tardar entre 3 y 10 minutos la primera vez. Al terminar debe mostrar algo como `added 400 packages` y volver al cursor.

> **Si npm ci falla**
>
> Pruebe `npm install` . Si el error menciona la red o un proxy, revise la conexión a internet. Si menciona permisos, abra PowerShell como administrador.

2. Crear las tablas

```
npm run db:schema
```

Debe ver:

```
Aplicando esquema desde sql/schema.sql ...
Esquema aplicado correctamente.
```

El esquema crea **9 tablas**: usuarios, periodos académicos, grupos, integrantes, bitácoras, servicios institucionales, configuración, plantillas y solicitudes. Se puede ejecutar más de una vez sin borrar datos.

3. Cargar los datos de ejemplo

```
npm run db:seed -- --confirm-seed
```

Debe ver una salida parecida a esta:

```
Ejecutando seed de la base de datos...
Periodo académico 2026-I listo (id=1)
  + creado cmendoza@iunav.edu.ve (admin)
  + creado mlopez@iunav.edu.ve (coordinador)
  + creado jperez@iunav.edu.ve (tutor)
  ...
  + configuracion_sistema creada (90h directas + 30h impacto)
  + 5 plantillas documentales
  + grupo Mantenimiento de Red Comunitaria ...
  + 12 bitácoras comunitarias y 10 servicios sociales/ambientales
Seed completado.
```

El seed también se puede ejecutar más de una vez: no duplica usuarios ni grupos. La bandera `--confirm-seed` es obligatoria porque las cuentas de ejemplo tienen contraseñas conocidas y públicas; así se evita cargarlas por accidente en un servidor real.

4. Si algo falla en este paso

Mensaje	Qué hacer
`database "app_iunav" does not exist`	Falta crear la base. Vuelva al paso 4.
`password authentication failed for user "postgres"`	La contraseña de `DATABASE_URL` no es correcta. Corrija **`server\.env`**.
`self-signed certificate`	Falta `DB_SSL=false` en **`server\.env`**.
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL no está corriendo o el puerto es distinto. Inicie el servicio.
`Este seed crea usuarios de prueba...`	Faltó la bandera. Ejecute `npm run db:seed -- --confirm-seed`.

9. Paso 7: levantar la API

La API debe quedar corriendo en una terminal mientras use el sistema. Abra **PowerShell** y ejecute:

```
cd C:\IUNAV\servicio-comunitario-iunav\server
npm run dev
```

Debe ver mensajes parecidos a estos:

```
> iunav-servicio-comunitario-server@0.1.0 dev
> tsx watch src/index.ts

>> API IUNAV Servicio Comunitario escuchando en http://127.0.0.1:4000
>> Health check: http://127.0.0.1:4000/api/health
```

> **Deje esta terminal abierta**
>
> No cierre la ventana mientras use la web. Si la cierra, la API se detiene y el navegador no podrá cargar datos.

Comprobar la API

Abra el navegador en:

```
http://localhost:4000/api/health
```

Debe ver un JSON parecido a este:

```
{"ok":true,"baseDeDatos":true,"servicio":"IUNAV Servicio Comunitario API","hora":"..."}
```

`ok:true` y `baseDeDatos:true` significan que la API está viva y que PostgreSQL responde. Si ve `503` o `baseDeDatos:false`, la API está arriba pero no puede hablar con la base: revise el servicio de PostgreSQL y **`server\.env`**.

Si el puerto 4000 está ocupado

El mensaje suele ser `EADDRINUSE`. Identifique el proceso y ciérrelo:

```
netstat -ano | findstr :4000
taskkill /PID NUMERO_DE_PID /F
```

Reemplace `NUMERO_DE_PID` por el número que aparece al final de la línea. Si no está seguro de qué programa es, cierre otras terminales o aplicaciones y vuelva a intentar. También puede cambiar `PORT` en **`server\.env`**, pero entonces tendrá que ajustar la web.

10. Paso 8: levantar la web

Abra una **segunda** terminal de PowerShell. La primera debe seguir con la API corriendo. En la nueva terminal ejecute:

```
cd C:\IUNAV\servicio-comunitario-iunav\web
npm ci
npm run dev
```

La primera instalación de la web también tarda unos minutos. Al terminar debe ver algo parecido a esto:

```
▲ Next.js 16.3.5
- Local: http://localhost:3000
- Network: http://192.168.x.x:3000
✓ Ready in 2.1s
```

Abra el navegador en:

```
http://localhost:3000
```

> **Use localhost, no 127.0.0.1, para abrir la web**
>
> La API solo acepta peticiones del origen `http://localhost:3000`. Si abre `http://127.0.0.1:3000`, el navegador puede bloquear las peticiones por CORS y el inicio de sesión no funcionará. Use siempre `localhost`.

Archivo opcional web\.env.local

Normalmente no hace falta. Si la web no encuentra la API, cree el archivo **`web\.env.local`** con esta línea:

```
NEXT_PUBLIC_API_URL=http://localhost:4000
```

Para crearlo desde PowerShell:

```
cd C:\IUNAV\servicio-comunitario-iunav\web
Copy-Item .env.example .env.local
notepad .env.local
```

Guarde y reinicie la web con `Ctrl` + `C` y luego `npm run dev`. Este archivo tampoco se sube al repositorio.

Si el puerto 3000 está ocupado

```
netstat -ano | findstr :3000
taskkill /PID NUMERO_DE_PID /F
```

Si decide usar otro puerto para la web, tendrá que cambiar `FRONTEND_URL` en **`server\.env`** y reiniciar la API. Para la demostración, lo más simple es dejar la web en 3000.

> **La primera carga puede tardar**
>
> Next.js compila las páginas la primera vez que se abren. Es normal que el navegador tarde unos segundos y que la terminal muestre actividad. No cierre nada.

11. Paso 9: verificar con los cuatro roles

Cuando la web abra en `http://localhost:3000`, inicie sesión con las cuentas de ejemplo. Son cuentas de demostración: no las use en un sistema real.

Rol	Correo	Contraseña
Administrador	`cmendoza@iunav.edu.ve`	`admin123`
Coordinador	`mlopez@iunav.edu.ve`	`coord123`
Tutor	`jperez@iunav.edu.ve`	`tutor123`
Estudiante	`gmartinez@iunav.edu.ve`	`est123`

Recorrido de 10 minutos

1. Entre como **administrador**. Debe ver el tablero con datos y el menú completo.
2. Abra **Gestión de Usuarios**. Debe aparecer la lista con los usuarios de ejemplo.
3. Abra **Grupos y Proyectos**. Debe ver los proyectos creados por el seed.
4. Cierre sesión y entre como **coordinador**. Debe ver el mismo menú que el administrador.
5. Cierre sesión y entre como **tutor**. Abra **Validación** y revise las bitácoras pendientes de sus grupos.
6. Cierre sesión y entre como **estudiante**. Abra **Bitácora** y revise las metas de 90 horas directas y 30 de impacto.
7. Genere un reporte desde el módulo **Reportes** con el usuario administrador o coordinador.
8. Si algo no coincide, revise la sección 13 antes de cambiar la instalación.

Qué debería ver cada rol en el menú

Rol	Módulos visibles
Administrador	Dashboard, Registro de Usuarios, Gestión de Usuarios, Grupos y Proyectos, Configuraciones, Reportes, Mi perfil.
Coordinador	Dashboard, Registro de Usuarios, Gestión de Usuarios, Grupos y Proyectos, Configuraciones, Reportes, Mi perfil.
Tutor	Dashboard, Grupos y Proyectos, Validación, Reportes, Mi perfil.
Estudiante	Dashboard, Bitácora Comunitario, Servicio Social y Ambiental, Mi perfil.

> **Los datos son de demostración**
>
> Puede registrar, editar y validar información. Todo queda guardado en PostgreSQL y se conserva al cerrar y volver a abrir el sistema. Si quiere volver a los datos iniciales, vea la sección 14.

12. Uso diario: iniciar, detener y reiniciar

Iniciar el sistema

Después de la primera instalación, solo necesita dos terminales:

```
# Terminal 1 - API
cd C:\IUNAV\servicio-comunitario-iunav\server
npm run dev

# Terminal 2 - Web
cd C:\IUNAV\servicio-comunitario-iunav\web
npm run dev
```

Espere a que la API diga que está escuchando y a que la web diga `Ready`. Luego abra `http://localhost:3000`.

Detener el sistema

1. En la terminal de la web, presione `Ctrl` + `C`.
2. En la terminal de la API, presione `Ctrl` + `C`.
3. Puede cerrar las dos ventanas.

PostgreSQL puede seguir corriendo como servicio de Windows; no hace falta detenerlo cada vez. Si desea apagarlo, use **Servicios** o, en PowerShell como administrador:

```
Stop-Service postgresql-x64-17
```

Para volver a iniciarlo:

```
Start-Service postgresql-x64-17
```

Reiniciar la computadora

- PostgreSQL arranca solo con Windows, salvo que se haya desactivado el inicio automático.
- Después de encender, abra las dos terminales y ejecute `npm run dev` en cada una.
- No hace falta repetir `npm ci`, `db:schema` ni `db:seed`.

Qué se conserva y qué no

Elemento	Se conserva al cerrar	Dónde está
Usuarios, grupos, bitácoras y servicios	Sí	Base `app_iunav` en PostgreSQL
Archivos subidos (evidencias, firmas, documentos)	Sí	**`server\uploads\`**
Configuración de la conexión	Sí	**`server\.env`**
API y web encendidas	No	Hay que ejecutar `npm run dev` otra vez

> **Si algo queda raro**
>
> Cierre las dos terminales con `Ctrl` + `C`, abra dos nuevas y vuelva a ejecutar `npm run dev` en **`server\`** y **`web\`**. No reinstale nada.

13. Si algo falla

Busque el mensaje exacto en la tabla. Si no aparece, revise la terminal de la API y la de la web: casi siempre el error real está allí.

Síntoma o mensaje	Causa probable	Solución
`node` o `npm` no se reconoce	Node no está instalado o el PATH no se actualizó.	Cierre y abra una terminal nueva. Si sigue, reinstale Node LTS con la opción *Add to PATH*.
`psql` no se reconoce	La carpeta `bin` de PostgreSQL no está en el PATH.	Use **SQL Shell** del menú Inicio o la ruta `C:\Program Files\PostgreSQL\17\bin\psql.exe`.
`password authentication failed for user "postgres"`	La contraseña de **`server\.env`** no coincide.	Corrija `DATABASE_URL`. Si olvidó la contraseña, reinstale PostgreSQL o pida ayuda.
`database "app_iunav" does not exist`	No se creó la base.	Vuelva al paso 4 y cree `app_iunav`.
`self-signed certificate` o error de SSL	Falta `DB_SSL=false`.	Agregue o corrija esa línea en **`server\.env`** y reinicie la API.
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL no está corriendo o usa otro puerto.	Inicie el servicio con `Start-Service postgresql-x64-17` o revise el puerto.
`EADDRINUSE: address already in use :::4000`	Otro proceso usa el puerto 4000.	`netstat -ano | findstr :4000` y `taskkill /PID PID /F`.
`EADDRINUSE: address already in use :::3000`	Otro proceso usa el puerto 3000.	`netstat -ano | findstr :3000` y `taskkill /PID PID /F`.
`relation "usuarios" does not exist`	No se aplicó el esquema.	Ejecute `npm run db:schema` desde **`server\`**.
El seed se niega a ejecutarse	Falta la bandera de confirmación.	Use `npm run db:seed -- --confirm-seed`.
La web abre pero el login falla o queda cargando	La API está apagada, en otro puerto o el origen no coincide.	Compruebe `/api/health`; abra la web con `localhost` y revise `FRONTEND_URL`.
La web muestra una página en blanco o un error de compilación	Caché de Next corrupta o versión de Node incorrecta.	Cierre la web, borre **`web\.next`** y ejecute `npm run dev` otra vez.
`permission denied to create extension "pgcrypto"`	Se conectó con un usuario sin permisos de superusuario.	Use el usuario `postgres` en `DATABASE_URL`.
El puerto 5432 está ocupado	Ya hay otro PostgreSQL instalado.	Use ese servidor, o instale este en 5433 y cambie el puerto en `DATABASE_URL`.
El antivirus bloquea npm o PostgreSQL	Protección en tiempo real sobre la carpeta del proyecto.	Agregue una exclusión para **`C:\IUNAV`** o permita Node y PostgreSQL.

Comandos útiles para diagnosticar

```
# Versiones instaladas
node -v
npm -v

# Servicio de PostgreSQL
Get-Service postgresql*

# Puertos ocupados
netstat -ano | findstr :3000
netstat -ano | findstr :4000
netstat -ano | findstr :5432

# Salud de la API (con la API corriendo)
Invoke-RestMethod http://localhost:4000/api/health
```

> **Qué enviar si pide ayuda**
>
> Envíe el número del paso donde falló, una captura de la ventana completa, el mensaje exacto, la salida de `node -v` y `npm -v`, y si el servicio de PostgreSQL está en `Running`. No envíe **`server\.env`** ni su contraseña.

14. Reiniciar los datos y desinstalar

Volver a los datos de ejemplo

Esto borra todo lo que haya creado y deja la base como recién instalada. Úselo solo si quiere empezar de cero.

1. Detenga la API con `Ctrl` + `C`.
2. Abra pgAdmin y conéctese a PostgreSQL 17.
3. Haga clic derecho sobre `app_iunav` y elija **Delete/Drop**. Confirme.
4. Cree otra vez la base `app_iunav` como en el paso 4.
5. En **`server\`**, ejecute:

```
cd C:\IUNAV\servicio-comunitario-iunav\server
npm run db:schema
npm run db:seed -- --confirm-seed
npm run dev
```

También puede hacerlo con SQL, conectado a la base `postgres`:

```
DROP DATABASE app_iunav WITH (FORCE);
CREATE DATABASE app_iunav;
```

> **Esta operación borra usuarios, grupos, bitácoras y servicios**
>
> No hay forma de recuperarlos salvo que tenga un respaldo. Si solo quiere corregir un dato, use la interfaz del sistema, no borre la base.

Desinstalar todo

1. Detenga la web y la API con `Ctrl` + `C`.
2. Desinstale Node.js desde **Configuración** > **Aplicaciones**.
3. Desinstale PostgreSQL desde **Configuración** > **Aplicaciones**.
4. Si el desinstalador pregunta por la carpeta de datos, decida si quiere conservarla. Si la borra, se pierden todos los datos.
5. Borre la carpeta **`C:\IUNAV`**.

Los datos de PostgreSQL no viven dentro de la carpeta del proyecto. Viven en la carpeta de datos del servidor, normalmente **`C:\Program Files\PostgreSQL\17\data`**. Por eso, borrar solo el proyecto no borra la base.

Respaldo opcional

Si quiere guardar una copia antes de desinstalar, desde PowerShell:

```
& "C:\Program Files\PostgreSQL\17\bin\pg_dump.exe" -U postgres -d app_iunav -f
"$env:USERPROFILE\Desktop\respaldo-iunav.sql"
```

Le pedirá la contraseña de `postgres`. El respaldo queda en el escritorio, fuera de la carpeta que va a borrar.

15. Anexo: estructura, puertos y comandos

Qué hay en cada carpeta

```
servicio-comunitario-iunav\
  docs\      manuales y guías en HTML y PDF
  local\     kit automático: NO se usa en este manual
  server\    API (Express + TypeScript)
    .env.example   plantilla de configuración
    sql\schema.sql esquema de las 9 tablas
    src\           código de la API
    uploads\       archivos subidos (se crea al usarlo)
  web\       interfaz (Next.js + React)
  README.md  resumen del proyecto
```

Puertos

Servicio	Puerto	Se abre en el navegador
Web	`3000`	Sí: `http://localhost:3000`
API	`4000`	Solo para diagnóstico: `http://localhost:4000/api/health`
PostgreSQL	`5432`	No. Solo lo usa la API.

Comandos de referencia

Comando	Dónde	Qué hace
`npm ci`	server\ y web\	Instala las dependencias exactas del proyecto.
`npm run db:schema`	server\	Crea o actualiza las tablas.
`npm run db:seed -- --confirm-seed`	server\	Carga usuarios, grupos y datos de ejemplo.
`npm run dev`	server\ y web\	Arranca la API o la web en modo desarrollo.
`npm run build`	server\ y web\	Compila para producción (no es necesario para esta demostración).
`npm test`	server\	Ejecuta las pruebas automáticas; requiere `app_iunav_test`.

Documentación complementaria

- **Guía de usuario:** qué puede hacer cada rol, paso a paso.
- **Flujo de uso:** el ciclo completo del programa por etapas.
- **README del proyecto:** resumen técnico y enlaces a los manuales.

Ficha técnica

Interfaz	Next.js 16, React 19, Tailwind CSS 4
API	Node.js 24 LTS, Express 4, TypeScript, Zod
Base de datos	PostgreSQL 17, 9 tablas
Seguridad	JWT, permisos por rol, rate limit en el login
Endpoints	48 endpoints para 4 roles
Pruebas	449 pruebas automatizadas (Vitest + Supertest)
Documentación	**`docs\`** (HTML y PDF)

> **No use estas cuentas ni esta configuración en producción**
>
> Las contraseñas del seed son públicas y el sistema está pensado para demostración local. Para un servidor real hay que cambiar las cuentas, rotar el `JWT_SECRET` y seguir el manual de despliegue correspondiente.
